Les méthodes spéciales

Tous droits réservés à STUDI - Reproduction interdite

Table des matières

I. Introduction aux méthodes et différence entre les méthodes et les fonctions **3**

A. Les fonctions en Python 3

B. Les méthodes en Python 3

C. Différences entre méthodes et fonctions 4

D. Exercice 5

II. Les méthodes spéciales et leur portée **5**

A. Que sont les méthodes spéciales ? 5

B. La surcharge des méthodes spéciales 6

C. La portée des variables et fonctions 8

D. Exercice : Quiz 10

III. L'essentiel **10**

IV. Auto-évaluation **11**

A. Exercice 11

B. Test 11

Solutions des exercices **12**

Tous droits réservés à STUDI - Reproduction interdite

I. Introduction aux méthodes et différence entre les méthodes et les fonctions

A. Les fonctions en Python

Définition **Les fonctions**

En programmation, une fonction est un ensemble de déclarations qui accomplissent une tâche spécifique. Les fonctions fournissent une manière de rassembler des fragments de code pour qu'ils soient réutilisables et organisés. En Python, une fonction peut être définie en utilisant le mot-clé « *def* », suivi du nom de la fonction, d'une paire de parenthèses « *()* » et de deux-points « *:* ». Les déclarations qui composent la fonction sont écrites sur les lignes suivantes, indentées.

Rappel **L'indentation est importante en Python**

Pour rappel, la syntaxe de Python possède une particularité : l'indentation du code est fondamentale. En effet, les lignes ne se finissent pas par des « *;* » comme c'est le cas avec beaucoup d'autres langages de programmation. Par ailleurs, les blocs d'instructions ne sont pas encerclés par des accolades « *{…}* ». L'indentation est donc le seul moyen, pour l'interpréteur, d'exécuter correctement notre code.

Exemple

```
1 def ma_fonction():
2     print("Ceci est une fonction!")
3
4 ma_fonction()   # Affiche : Ceci est une fonction!
```

B. Les méthodes en Python

Définition **Les méthodes**

Une méthode est une fonction qui est associée à un objet particulier. En Python, cela signifie généralement que la méthode est définie dans une classe et est destinée à travailler avec les instances de cette classe. Dans la définition d'une méthode, le premier paramètre est généralement « *self* », qui est une référence à l'instance de l'objet qui est en train d'appeler la méthode.

Exemple

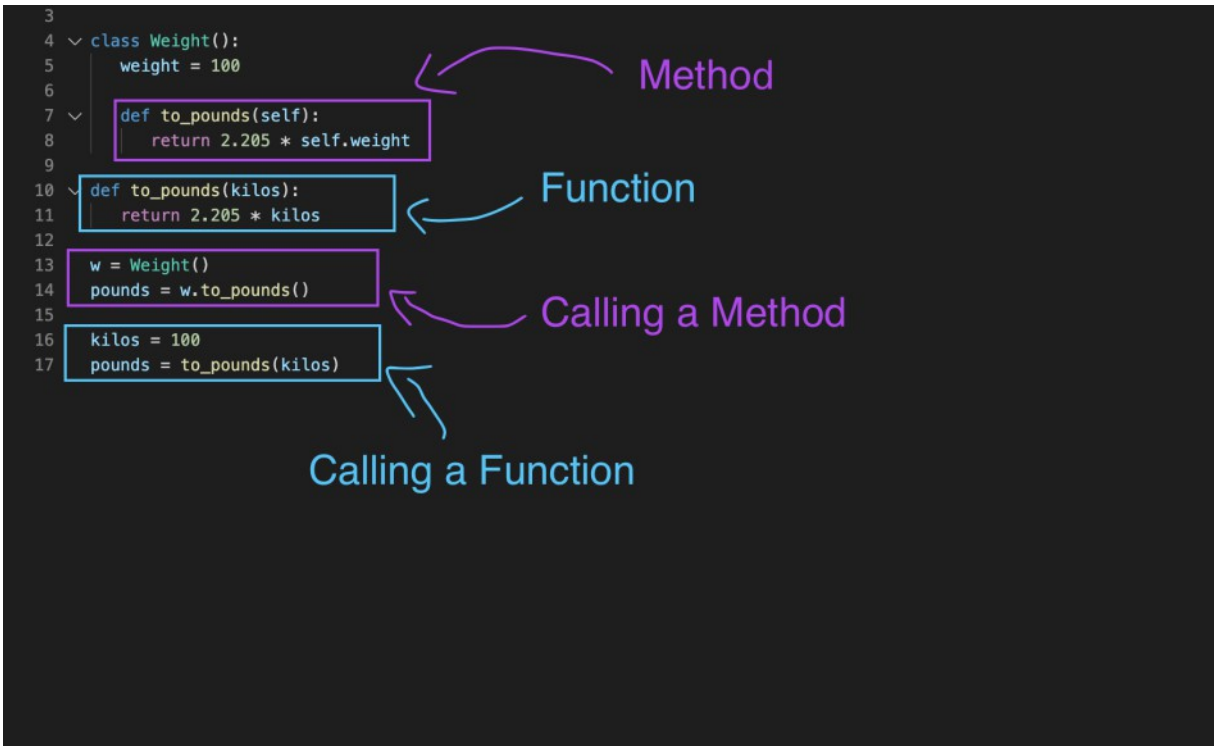
En Python, une méthode est définie à l'intérieur d'une classe et elle est destinée à être appelée par une instance de cette classe. Les méthodes peuvent accéder et modifier l'état de l'instance de la classe dont elles proviennent. Par exemple :

```
1 class MaClasse:
2     def ma_methode(self):
3         print("Ceci est une méthode!")
```

Dans cet exemple, ma_methode est une méthode de la classe MaClasse. Pour l'appeler, vous devez d'abord créer une instance de MaClasse, puis appeler la méthode sur cette instance :

```
1 mon_objet = MaClasse()
2 mon_objet.ma_methode() # Affiche : Ceci est une méthode!
```

C. Différences entre méthodes et fonctions



Source : Codingem[1]

La principale différence entre les méthodes et les fonctions en Python réside dans le fait que les méthodes sont associées à des objets (instances de classe), tandis que les fonctions sont indépendantes. Cela signifie que les méthodes peuvent manipuler l'état d'un objet, tandis que les fonctions ne le peuvent pas.

L'autre différence majeure est que le premier paramètre d'une méthode est toujours « *self* », qui est une référence à l'instance de la classe, tandis que les fonctions n'ont pas de paramètre « *self* ».

Souvent, les méthodes sont considérées comme étant un type particulier de fonction. C'est pourquoi certaines personnes utilisent donc toujours le terme « *fonction* », même si il s'agit en fait d'une méthode.

Exemple

```python
# Définition d'une fonction
def additionner(a, b):
    return a + b

# Utilisation de la fonction
resultat = additionner(5, 7)
print(resultat)         # Affiche : 12

# Définition d'une classe avec une méthode
class Calculateur :
    def __init__(self, valeur):
        self.valeur = valeur

def ajouter(self, autre_valeur):
    return self.valeur + autre_valeur

# Utilisation de la méthode
```

1. https://www.codingem.com/python-what-is-the-difference-between-a-function-and-a-method/

Tous droits réservés à STUDI - Reproduction interdite

```
18 calc = Calculateur(5)
19 resultat = calc.ajouter(7)
20 print(resultat)         # Affiche : 12
```

Dans ces exemples, nous avons d'abord défini une fonction « *additionner* » qui prend deux arguments et renvoie leur somme. Ensuite, nous avons défini une classe « *Calculateur* » avec une méthode « *ajouter* » qui prend un autre argument et renvoie la somme de cet argument et de la valeur stockée dans l'instance de la classe. Notez que dans la méthode « *ajouter* », nous avons utilisé « *self.valeur* » pour accéder à la valeur stockée dans l'instance de la classe.

Complément **L'importance des fonctions et des méthodes en programmation**

Les fonctions et les méthodes sont essentielles en programmation, car elles permettent la réutilisation de code et la modularité. L'écriture de code en petits blocs réutilisables rend le code plus lisible, plus facile à comprendre et à maintenir.

Réutilisation de code :

Plutôt que de répéter le même code à plusieurs endroits, vous pouvez encapsuler le code dans une fonction ou une méthode et l'appeler chaque fois que nécessaire.

Modularité :

Les fonctions et les méthodes permettent de diviser un programme complexe en petites parties gérables, appelées « *modules* ». Chaque fonction ou méthode représente un module ou une fonctionnalité unique.

D. Exercice

Question

[solution n°1 p.13]

Supposons que nous travaillons sur l'élaboration d'un site web. Nous aurons 3 types d'utilisateurs : des simples visiteurs, des modérateurs de contenu, ainsi que des administrateurs. Ces catégories d'utilisateurs constituerons trois objets différentes, héritant tous d'une et même classe générique. Pour chaque type d'utilisateur (et donc chaque objet), nous devons pouvoir :

1. Ajouter un utilisateur (en donnant son adresse mail)
2. Supprimer un utilisateur (en donnant son adresse mail)
3. Compter le nombre total d'utilisateurs

Proposez une implémentation.

II. Les méthodes spéciales et leur portée

A. Que sont les méthodes spéciales ?

Introduction aux méthodes spéciales

Les méthodes spéciales, aussi connues sous le nom de « *dunder methods* » pour « *double underscore* », sont des méthodes intégrées à Python qui ont une signification particulière pour l'interpréteur. Les noms de ces méthodes sont encadrés par des doubles underscores, d'où leur surnom. Ces méthodes sont invoquées automatiquement dans certaines situations. Par exemple, lorsqu'une instance d'une classe est créée, la méthode spéciale « *__init__()* » est appelée.

Lorsqu'une instance de classe est convertie en chaîne de caractères pour l'affichage, la méthode « *__str__()* » est appelée. Ces méthodes ne sont généralement pas appelées de manière explicite lors de l'écriture du programme, mais plutôt de manière implicite par l'interpréteur, lors de l'exécution de notre programme.

Tous droits réservés à STUDI - Reproduction interdite

Qu'est ce qu'une méthode spéciale ?

Une méthode spéciale est une méthode ayant un rôle précis dans la vie d'une instance de classe. Ces méthodes sont reconnues par l'interpréteur Python grâce à leur nom, qui est toujours entouré de doubles underscores. Par exemple, la méthode spéciale « *__init__()* » est automatiquement appelée lorsqu'une instance de la classe est créée. Cette méthode est souvent utilisée pour initialiser les attributs de l'objet. Une autre méthode spéciale couramment utilisée est « *__str__ ()* » , qui est appelée par la fonction print() et par la méthode str() pour convertir l'objet en une chaîne de caractères.

La plupart des opérateurs mathématiques font également appel à des méthodes spéciales. Par exemple la simple addition (« + ») entre deux objets fait appel à la méthode spéciale « *__add__ ()* » relative à (et définie par) la classe des objets additionnés.

Exemple **Méthodes spéciales courantes**

Les méthodes spéciales les plus couramment utilisées en Python sont :

_init _() : cette méthode est appelée lorsqu'une instance d'une classe est créée. Elle est souvent utilisée pour initialiser les attributs de l'objet.

__str__() : cette méthode est utilisée pour fournir une représentation sous forme de chaîne de caractères de l'objet. Elle est appelée lorsque vous utilisez la fonction « str(obj) » ou lorsque vous essayez d'afficher l'objet avec la fonction « print(obj) ».

__len__() : cette méthode est appelée lorsque vous utilisez la fonction « *len(obj)* ». Elle doit renvoyer un entier qui représente la « *longueur* » ou le « *nombre d'éléments* » de l'objet.

Ces exemples ne sont qu'un petit aperçu des nombreuses méthodes spéciales disponibles en Python.

Utilisation des méthodes spéciales

Utiliser une méthode spéciale consiste à la définir dans votre classe, tout comme vous le feriez pour une méthode normale, mais en respectant la convention de nommage avec les doubles underscores. Par exemple, si vous souhaitez que votre objet puisse être affiché de manière significative avec la fonction « *print()* », vous pouvez définir la méthode __str__() dans votre classe. Ceci permettra à l'interpréteur de savoir comment convertir votre objet en chaine de caractères.

Les méthodes spéciales rendent votre code plus simple, pratique et intuitif, mais elles le rendent également beaucoup plus simple à lire.

Remarque **Cas d'utilisation spécifiques des méthodes spéciales**

Certaines méthodes spéciales ont des applications très précises. Par exemple, « *__init__()* » est utilisée pour initialiser les attributs d'une instance lors de sa création. La méthode « *__str__()* » est utilisée pour fournir une représentation lisible de l'objet lors de son affichage. « *__len__()* » est généralement utilisée pour indiquer la « *taille* » ou le « *nombre d'éléments* » d'un objet. Chaque méthode spéciale a une utilisation précise qui dépend des besoins de votre classe.

B. La surcharge des méthodes spéciales

La surcharge des méthodes spéciales est une technique courante en Python. Surcharger une méthode spéciale, c'est simplement en définir son comportement, qui sera propre à notre classe. Par hiérarchisation, la définition personnalisée de nos méthodes spéciales sera toujours prioritaire par rapport à l'implémentation « *de base* » des méthodes spéciales propres à Python.

Tous droits réservés à STUDI - Reproduction interdite

Exemple **Surcharge de la fonction __add__()**

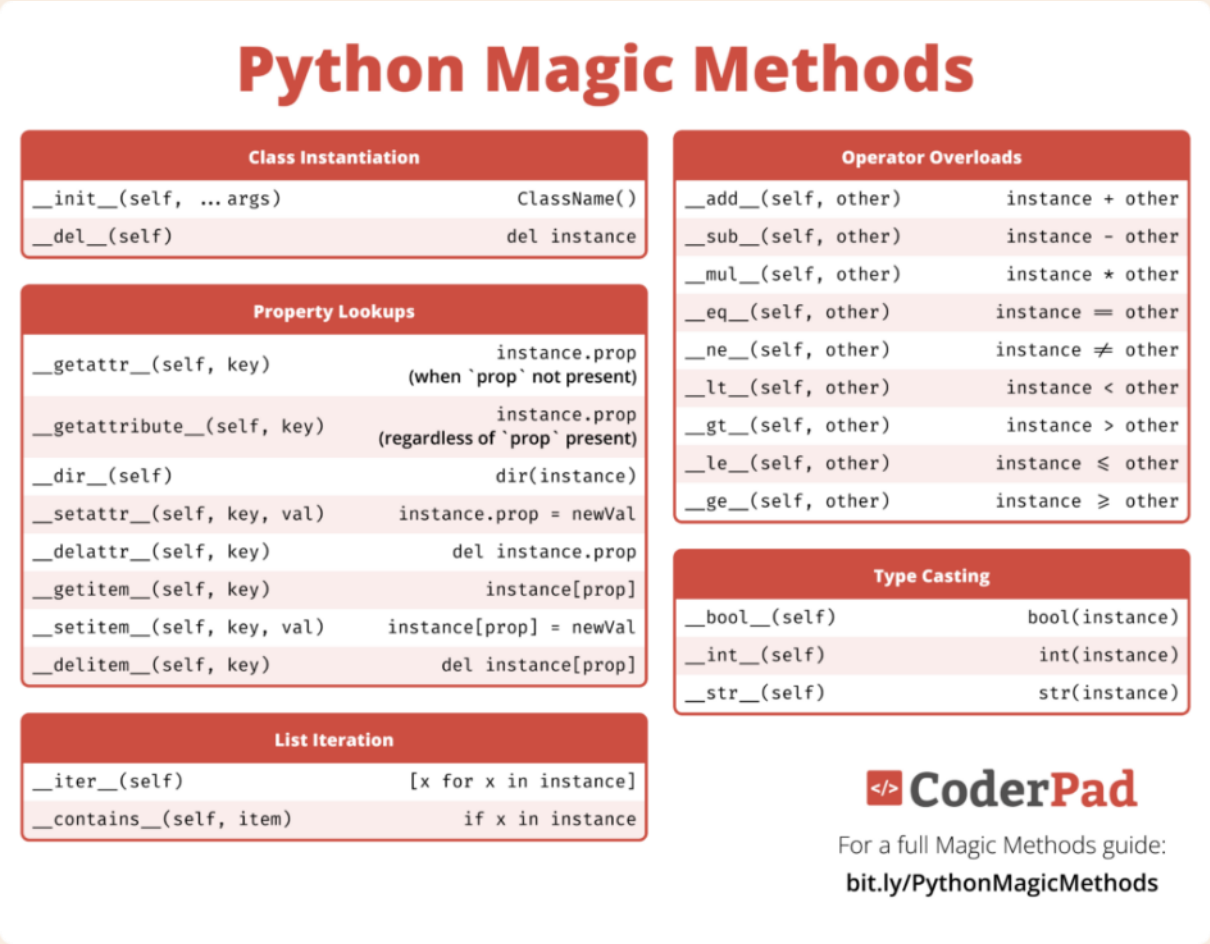
Reprenons l'exemple traité dans la vidéo précédente, ou nous avions une classe générique représentant un groupe d'utilisateurs, et 3 instances de cette classe : les visiteurs, les modérateurs ainsi que les administrateurs. Supposons qu'on souhaite pouvoir calculer la somme des utilisateurs de plusieurs groupes, en utilisant l'opérateur d'addition « + ». Pour ce faire, il faut simplement surcharger la méthode __add__() :

```
1 class UsersGroup:
2     usersArray = []
3     numberOfUsers = 0
4     isAdmin = False
5 
6     def AddUser(self, mailAddress):
7         self.usersArray.append(mailAddress)
8         self.numberOfUsers += 1
9 
10    def RemoveUser(self, mailAddress):
11        self.usersArray.remove(mailAddress)
12        self.numberOfUsers -= 1
13 
14    def GetNbrOfUsers(self):
15        return self.numberOfUsers
16 
17    def __add__(self, x):     # Surcharge de __add__()
18        return (self.numberOfUsers + x.numberOfUsers)
19 
20 
21 visitors = UsersGroup()
22 moderators = UsersGroup()
23 administrators = UsersGroup()
24 
25 # Ajoutons deux utilisateurs au groupe de modérateurs
26 moderators.AddUser("test1@mywebsite.com")
27 moderators.AddUser("test2@mywebsite.com")
28 
29 # Ajoutons un utilisateur au groupe des administrateurs
30 administrators.AddUser("test@mywebsite.com")
31 
32 sum = moderators + administrators
33 
34 print(sum)         # Imprime “3” à l'écran
```

Tous droits réservés à STUDI - Reproduction interdite

Complément **Les méthodes spéciales communes**

Le tableau suivant synthétise les méthodes spéciales les plus « *standard* ». Il en existe évidemment d'autres, celles-ci peuvent être consultées dans la documentation officielle : Python[2]



Source : CoderPad[3]

C. La portée des variables et fonctions

Les catégories de portée

Il existe trois principaux niveaux de portée en Python :

- Globale : une variable ou une méthode définie en dehors de tout contexte ou de toute fonction.

  Ces variables/méthodes/fonctions peuvent être appelées de n'importe quel endroit dans votre code,
- Locale : une variable ou une méthode définie à l'intérieur d'une fonction ou d'une méthode.

  Ces variables/méthodes/fonctions ne peuvent être appelées que depuis l'intérieur de votre fonction ou méthode. Elles n'existent pas en dehors de celle-ci.
- Classe : une variable ou une méthode définie à l'intérieur d'une classe.

  Les variables / méthodes de classe sont définies pour toutes les instances de celle-ci.

2. https://docs.python.org/3/reference/datamodel.html#special-method-names
3. https://coderpad.io/python-magic-methods-cheat-sheet/

Tous droits réservés à STUDI - Reproduction interdite

Certaines variables/méthodes de classe sont particulières. Il s'agit des variables ou méthodes statiques. Les méthodes statiques sont simplement précédées du mot clé « *@staticmethod* » lors de leur définition. Ces méthodes ou variables sont définies dans le cadre d'une classe, et sont partagées par toutes les instances héritant de cette classe, contrairement à des variables de classe classiques.

La plupart des méthodes spéciales sont locales. Cependant, certaines d'entre elles sont locales et statiques, comme c'est le cas par exemple de la méthodes spéciales __init__().

Exemple **La portée des variables et des fonctions**

Code de l'exemple :

```
1 authorizeNewUsers = False               # Variable globale
2
3 def sendWelcomeMessage(mailAddress):    # Fonction globale
4     content = mailAddress + " Vous avez été ajouté a la liste des utilisateurs du site !"
5     # sendMail(content, mailAddress)
6     print("Envoi de l'email... contenu : " + content)
7
8 class UsersGroup:
9     usersArray = []     # Variable d'instance
10    numberOfUsers = 0   # Variable d'instance
11    isAdmin = False     # Variable d'instance
12    totalUsers = 0      # Variable de classe statique
13
14    def AddUser(self, mailAddress):
15        self.usersArray.append(mailAddress)
16        self.numberOfUsers += 1
17        UsersGroup.totalUsers += 1          # On incrémente la variable statique
18        sendWelcomeMessage(mailAddress)     # Appel à la fonction globale
19
20    def RemoveUser(self, mailAddress):
21        self.usersArray.remove(mailAddress)
22        self.numberOfUsers -= 1
23
24    def GetNbrOfUsers(self):
25        return self.numberOfUsers
26
27    def __add__(self, x):
28        return (self.numberOfUsers + x.numberOfUsers)
29
30
31 visitors = UsersGroup()
32 moderators = UsersGroup()
33 administrators = UsersGroup()
34
35 moderators.AddUser("test1@mywebsite.com")
36 moderators.AddUser("test2@mywebsite.com")
37
38 administrators.AddUser("test@mywebsite.com")
39
40 print(moderators.numberOfUsers)
41 print(administrators.numberOfUsers)
42 print(visitors.numberOfUsers)
43
44 print(UsersGroup.totalUsers)
```

Tous droits réservés à STUDI - Reproduction interdite

D. Exercice : Quiz

[solution n°2 p.13]

Question 1

Qu'est-ce qu'une méthode spéciale en Python ?

- ☐ Une méthode que vous définissez vous-même pour réaliser une tâche spécifique
- ☐ Une méthode intégrée à Python qui a une signification particulière pour l'interpréteur Python
- ☐ Une méthode qui est toujours publique et accessible depuis n'importe où dans le code

Question 2

Qu'est-ce que « *surcharger* » une méthode spéciale ?

- ○ Appeler une méthode spéciale plus d'une fois
- ○ Modifier le comportement par défaut d'une méthode spéciale
- ○ Créer une nouvelle méthode spéciale qui n'existait pas auparavant

Question 3

À quoi sert la méthode spéciale `__len__()` en Python ?

- ○ Elle est appelée lorsque vous utilisez la fonction len(obj)
- ○ Elle est appelée pour initialiser les attributs d'une instance de classe
- ○ Elle est utilisée pour convertir une instance de classe en chaîne de caractères

Question 4

Qu'est-ce que la portée globale en Python ?

- ○ Une variable ou une méthode définie au niveau le plus haut de votre fichier Python
- ○ Une variable ou une méthode définie à l'intérieur d'une fonction ou dune méthode
- ○ Une variable définie dans une fonction englobante, mais utilisée dans une fonction imbriquée

Question 5

Comment une méthode spéciale est-elle généralement appelée ?

- ○ Directement par le développeur
- ○ Automatiquement par l'interpréteur Python lors de certaines opérations
- ○ En utilisant le mot-clé call

III. L'essentiel

Les méthodes et les fonctions sont la colonne vertébrale de toute application Python, elles permettent d'encapsuler la logique et d'assurer la réutilisabilité du code. Une compréhension claire de ces concepts est essentielle pour naviguer avec aisance dans les différentes facettes de la programmation en Python.

Les méthodes spéciales, parfois appelées « *dunder methods* » en raison de leur double soulignement (underscore), sont un outil puissant pour personnaliser le comportement des objets.

L'usage approprié des méthodes et des fonctions peut améliorer la lisibilité et l'efficacité de votre code, tout en réduisant les erreurs potentielles. De même, la connaissance des méthodes spéciales et de leur application vous permettra de profiter pleinement des caractéristiques uniques de Python en tant que langage de programmation.

Tous droits réservés à STUDI - Reproduction interdite

La compréhension de la portée est également indispensable pour gérer efficacement les variables et les méthodes au sein de votre code. En distinguant clairement les portées globale, locale et statique, vous pouvez écrire un code plus sécurisé et plus robuste.

IV. Auto-évaluation

A. Exercice

Question 1

[solution n°3 p.14]

Créez une classe Point avec deux attributs, x et y. Surchargez la méthode spéciale __str__() pour personnaliser la représentation sous forme de chaîne de caractères de vos objets Point. Lorsqu'on imprime un objet Point, le résultat doit être sous la forme « *(x, y)* ».

Question 2

[solution n°4 p.14]

Ajoutez une méthode spéciale __add__() à la classe Point pour surcharger l'opérateur « + ». Cette méthode doit permettre d'additionner les coordonnées de deux points pour obtenir un nouvel objet Point. Après avoir surchargé la méthode __add__(), l'addition de deux points p1 et p2 avec p1 + p2 doit renvoyer un nouvel objet Point dont les coordonnées sont la somme des coordonnées de p1 et p2.

B. Test

Exercice 1 : Quiz

[solution n°5 p.14]

Question 1

Qu'est-ce qu'une fonction en Python ?

- ○ Un objet complexe qui peut contenir d'autres objets
- ○ Un ensemble de tâches spécifiques encapsulées dans un ensemble de déclarations
- ○ Une tâche spécifique qui ne peut pas être réutilisée
- ○ Une tâche spécifique encapsulée dans une classe

Question 2

Qu'est-ce qu'une méthode en Python ?

- ○ Une fonction indépendante qui ne dépend d'aucun objet
- ○ Une fonction associée à un objet particulier
- ○ Une fonction qui ne peut pas être réutilisée
- ○ Une fonction qui ne peut pas être modifiée

Question 3

Qu'est-ce que __str__() en Python ?

- ○ C'est une méthode spéciale qui est utilisée pour convertir l'objet en une chaîne de caractères
- ○ C'est une fonction qui est utilisée pour convertir l'objet en un entier
- ○ C'est une méthode spéciale qui est appelée lorsque vous utilisez la fonction len(obj)
- ○ C'est une fonction qui est appelée lorsque vous essayez d'afficher un objet avec print(obj)

Question 4

Tous droits réservés à STUDI - Reproduction interdite

Qu'est-ce que la portée d'une méthode ?

- C'est le niveau d'accès qu'une fonction a aux variables et aux autres fonctions
- C'est le niveau de protection qu'une fonction a contre les modifications
- C'est le niveau de contrôle qu'une fonction a sur les autres fonctions
- C'est le niveau de complexité d'une fonction

Question 5

Comment la portée affecte-t-elle l'utilisation des fonctions ?

- La portée détermine comment une fonction peut être appelée
- La portée détermine quand une fonction peut être appelée
- La portée détermine pourquoi une fonction peut être appelée
- La portée détermine où une fonction peut être appelée

Solutions des exercices

Tous droits réservés à STUDI - Reproduction interdite

p. 5 Solution n°1

Réponse en vidéo

Exercice p. 10 Solution n°2

Question 1

Qu'est-ce qu'une méthode spéciale en Python ?

- [x] Une méthode que vous définissez vous-même pour réaliser une tâche spécifique
- [x] Une méthode intégrée à Python qui a une signification particulière pour l'interpréteur Python
- [ ] Une méthode qui est toujours publique et accessible depuis n'importe où dans le code

Question 2

Qu'est-ce que « *surcharger* » une méthode spéciale ?

- ○ Appeler une méthode spéciale plus d'une fois
- ◉ Modifier le comportement par défaut d'une méthode spéciale
- ○ Créer une nouvelle méthode spéciale qui n'existait pas auparavant

Question 3

À quoi sert la méthode spéciale `__len__()` en Python ?

- ◉ Elle est appelée lorsque vous utilisez la fonction len(obj)
- ○ Elle est appelée pour initialiser les attributs d'une instance de classe
- ○ Elle est utilisée pour convertir une instance de classe en chaîne de caractères

Question 4

Qu'est-ce que la portée globale en Python ?

- ◉ Une variable ou une méthode définie au niveau le plus haut de votre fichier Python
- ○ Une variable ou une méthode définie à l'intérieur d'une fonction ou dune méthode
- ○ Une variable définie dans une fonction englobante, mais utilisée dans une fonction imbriquée

Question 5

Comment une méthode spéciale est-elle généralement appelée ?

- ○ Directement par le développeur
- ◉ Automatiquement par l'interpréteur Python lors de certaines opérations
- ○ En utilisant le mot-clé call

Tous droits réservés à STUDI - Reproduction interdite

p. 11 Solution n°3

```python
class Point:
  def __init__(self, x, y):
    self.x = x
    self.y = y

  def __str__(self):
    return f"({self.x}, {self.y})"
```

p. 11 Solution n°4

```python
class Point:
  def __init__(self, x, y):
    self.x = x
    self.y = y

  def __str__(self):
    return f"({self.x}, {self.y})"

  def __add__(self, other):
    if isinstance(other, Point):
      return Point(self.x + other.x, self.y + other.y)
    else:
      raise TypeError("You can only add two Points together")

# Nous pouvons désormais utiliser l'opérateur '+' pour ajouter les coordonnées de deux points.

# Création de deux points
p1 = Point(1, 2)
p2 = Point(3, 4)

# Addition des coordonnées de deux points
p3 = p1 + p2

print("Le résultat de l'addition de {p1} et {p2} est {p3}.")
```

Exercice p. 11 Solution n°5

Question 1

Qu'est-ce qu'une fonction en Python ?

- ○ Un objet complexe qui peut contenir d'autres objets
- ◉ Un ensemble de tâches spécifiques encapsulées dans un ensemble de déclarations
- ○ Une tâche spécifique qui ne peut pas être réutilisée
- ○ Une tâche spécifique encapsulée dans une classe

En Python, une fonction est un bloc de code réutilisable qui effectue une action spécifique. Les fonctions fournissent un moyen de structurer le code pour permettre la réutilisation, la modularité et l'organisation.

Tous droits réservés à STUDI - Reproduction interdite

Question 2

Qu'est-ce qu'une méthode en Python ?

- ○ Une fonction indépendante qui ne dépend d'aucun objet
- ◉ Une fonction associée à un objet particulier
- ○ Une fonction qui ne peut pas être réutilisée
- ○ Une fonction qui ne peut pas être modifiée

En Python, une méthode est similaire à une fonction, mais elle est associée à un objet particulier. Chaque objet d'une classe a accès aux méthodes de cette classe, qui peuvent manipuler les données contenues dans l'objet.

Question 3

Qu'est-ce que __str__() en Python ?

- ◉ C'est une méthode spéciale qui est utilisée pour convertir l'objet en une chaîne de caractères
- ○ C'est une fonction qui est utilisée pour convertir l'objet en un entier
- ○ C'est une méthode spéciale qui est appelée lorsque vous utilisez la fonction len(obj)
- ○ C'est une fonction qui est appelée lorsque vous essayez d'afficher un objet avec print(obj)

La méthode spéciale __str__() en Python est une méthode intégrée qui est utilisée pour convertir un objet en une chaîne de caractères. Elle est généralement utilisée pour fournir une représentation lisible de l'objet.

Question 4

Qu'est-ce que la portée d'une méthode ?

- ◉ C'est le niveau d'accès qu'une fonction a aux variables et aux autres fonctions
- ○ C'est le niveau de protection qu'une fonction a contre les modifications
- ○ C'est le niveau de contrôle qu'une fonction a sur les autres fonctions
- ○ C'est le niveau de complexité d'une fonction

La portée d'une méthode détermine où dans le code une fonction peut accéder à certaines variables ou à d'autres fonctions. Par exemple, une fonction peut avoir accès aux variables locales définies à l'intérieur de la fonction, mais pas aux variables définies en dehors de cette fonction.

Question 5

Comment la portée affecte-t-elle l'utilisation des fonctions ?

- ○ La portée détermine comment une fonction peut être appelée
- ○ La portée détermine quand une fonction peut être appelée
- ○ La portée détermine pourquoi une fonction peut être appelée
- ◉ La portée détermine où une fonction peut être appelée

La portée d'une fonction affecte où elle peut être appelée dans le code. Par exemple, une fonction définie à l'intérieur d'une classe (dans ce cas, cette fonction est une méthode) peut généralement être appelée par les autres méthodes de la classe, mais pas en dehors de la classe.

Tous droits réservés à STUDI - Reproduction interdite

Tous droits réservés à STUDI - Reproduction interdite